

Test Report:

Student: Sumayyah Gul-Shafi [M00995868]

Module: MSO2700 Software Design

Task: Tree Sort Implementation and Testing

Purpose:

The purpose of this testing is to verify that the tree sort method correctly sorts a list of integers into ascending order.

Required Test Case:

Input: (10, 5, 8, 3, 1, 5)

Expected Output: (1, 3, 5, 5, 8, 10)

Result: The method returned the correct sorted list.

Additional Test Cases:

The following additional tests were carried out to ensure robustness:

- Empty list → returns empty list
- Single element → unchanged
- Negative numbers → correctly sorted
- Duplicate values → handled correctly
- Already sorted list → unchanged
- Reverse order → correctly sorted
- Null input → handled safely

All tests passed successfully.

Test Evidence:



```
1 public class TreeSortRunner {
2     public static void main(String[] args) {
3         // Creating an instance of the test class
4         TreeSortTest testRunner = new TreeSortTest();
5
6         try {
7             // Running all the JUnit test methods manually:
8             testRunner.testRequireList();
9             testRunner.testEmptyList();
10            testRunner.testSingleElement();
11            testRunner.testNegativeNumbers();
12            testRunner.testAllDuplicates();
13            testRunner.testAlreadySorted();
14            testRunner.testReverseOrder();
15            testRunner.testNullInput();
16
17            // If no exceptions occur, all tests have passed successfully
18            System.out.println("All tests passed successfully.");
19        }
20        catch (AssertionError e) {
21            // If any tests fail, display the failure message
22            System.out.println("A test failed: " + e.getMessage());
23        }
24    }
25 }
26
27 }
```

Language version: JDK 25.0.2 ☒ Interactive Mode

Input arguments

Output Generated files

All tests passed successfully.

Compiled and executed in 1.946 sec(s)

Conclusion:

The tree sort implementation works correctly for the required input and handles a variety of edge cases successfully.